SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Dry Run Evaluation

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO1 – Apply (BL3)	**Assessment:**	Assessment Tool 2 – Dry Run Evaluation
Weightage:	35% of CIA	**Duration / Marks:**	60 mins / 100 marks
CO1 Statement:	Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)		
Student Name:	N.Sai Sankar	**Reg. No.:**	192311187
Section / Batch:	CSE	**Date:**	10/10/2026

Code of Conduct

I N.Sai Sankar 192311187 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N.Sai Sankar

Instructions

- Read each question carefully; answer all four questions (25 marks each, 100 in total).
- Show every dry-run step: the initial queue, stack, priority queue or belief state, each iteration, your assumptions and a clear final answer, and finish with the interpretation asked for.
- Follow the conventions stated in each question (alphabetical expansion, explored set, goal test on removal from the frontier, tie-breaking) and present tables neatly.
- Marks follow the Dry Run Evaluation rubric: understanding of the search problem, state expansion and dry run execution, application of the search algorithm, accuracy of the solution path, and presentation and logical reasoning.
- Upload in GitHub and GCR (question paper with Reg. No., Name, answers, rubrics and evaluation page at the end). Duration: 1 hour (15 minutes per question).

Section / Topic	Focus	Question Nos.
1. Foundations and History of AI	Four approaches to AI, Turing Test, eras and milestones	**Q2**
2. Intelligent Agents: Agents and Environments	Agent, percepts, actions, agent function and program, PEAS	**Q1**
3. The Concept of Rationality	Performance measure, percept sequence, rational vs omniscient	**Q1**
4. The Nature of Environments	Task-environment properties and their classification	**Q1**
5. The Structure of Agents	Simple reflex and model-based reflex agent programs	**Q1**
6. Problem-Solving Agents	Goal formulation, problem formulation, search, execution	**Q2**
7. Problem Formulation	States, initial state, actions, goal test, path cost; belief states	**Q2, Q4**
8. Searching for Solutions	Search graph, frontier, explored set, node expansion	**Q3**

9. Uninformed Search Strategies	Dry runs of BFS, DFS and uniform-cost search	**Q3**
10. Search with Partial Information	Sensorless problems, belief states, percept update	**Q4**

Annexure A – Dry Run Evaluation

Question 1: Agents, Environments and Rationality – Dry Run of Agent Programs

Topic: Intelligent agents, rationality, nature of environments, structure of agents | 15 minutes

A vacuum-cleaner agent lives in a world with two squares, A and B. At each time step it perceives [its location, status of its own square] and performs one action: Suck, Right, Left or NoOp. The agent starts in square A, both squares are initially Dirty, and dirt never reappears.

Performance measure over T = 6 time steps: +10 for every square cleaned (a Suck on a Dirty square) and −1 for every Left or Right move.

Agent X (simple reflex): if the current square is Dirty, Suck; otherwise move to the other square.

Agent Y (model-based reflex): keeps an internal state of the squares it knows to be clean. If the current square is Dirty, Suck; otherwise, if both squares are known clean, NoOp; otherwise move to the other square.

(a) Write the PEAS description (Performance measure, Environment, Actuators, Sensors) of this agent. **[4 Marks]**

(b) Classify the task environment on observability, number of agents, determinism, episodic/sequential, static/dynamic and discrete/continuous, with a one-phrase justification each. **[4 Marks]**

(c) Dry-run both agents for all 6 time steps. Complete the table (percept, action, and Y's internal state after the step). **[9 Marks]**

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-Clean Squares
1					
2					
3					
4					
5					
6					

(d) Compute the total performance of X and of Y over the 6 steps. **[3 Marks]**

(e) Interpret: which agent is rational? Both agents receive the identical percept [B, Clean] at step 4 – explain why they act differently, using the structure of the two agent programs, and distinguish a rational agent from an omniscient agent. **[5 Marks]**

QUESTION 1: ANSWER

(a) PEAS Description

PEAS Component	Description
Performance Measure	+10 for every square cleaned and −1 for every Left/Right movement
Environment	Two-square vacuum world containing squares A and B, initially both Dirty
Actuators	Suck, Left, Right and NoOp
Sensors	Sensor perceives the current location and status of the current square

Answer:

The vacuum-cleaner agent aims to maximize its performance by cleaning dirty squares while minimizing unnecessary movement.

(b) Task Environment Classification

Property	Classification	Justification
Observability	Partially observable	The agent only perceives its current location and current square's status
Number of Agents	Single-agent	Only one vacuum agent operates in the environment
Determinism	Deterministic	Each action has a predictable result
Episodic / Sequential	Sequential	Current actions affect the future state
Static / Dynamic	Static	Dirt does not reappear and the environment does not change by itself
Discrete / Continuous	Discrete	Locations, dirt states and actions are discrete

(c) Dry Run of Agent X and Agent Y

Initially:

- Location = **A**
- A = Dirty
- B = Dirty
- Y's known-clean set = **{}**

Dry-run table

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-Clean Squares
1	[A, Dirty]	Suck	[A, Dirty]	Suck	{A}
2	[A, Clean]	Right	[A, Clean]	Right	{A}
3	[B, Dirty]	Suck	[B, Dirty]	Suck	{A, B}
4	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}
5	[A, Clean]	Right	[B, Clean]	NoOp	{A, B}
6	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}

Explanation

Agent X:

After cleaning B, it has no memory of the fact that B is already clean. Therefore, whenever it sees a clean square, it simply moves to the other square.

Agent Y:

Agent Y remembers which squares are known to be clean. After both A and B are clean, it performs **NoOp** instead of moving unnecessarily.

(d) Total Performance

Agent X

Cleaning actions:

- Step 1: Suck on Dirty = **+10**
- Step 3: Suck on Dirty = **+10**

Movement actions:

- Step 2: Right = **−1**
- Step 4: Left = **−1**
- Step 5: Right = **−1**
- Step 6: Left = **−1**

Therefore,

$$Performance(X) = 10 + 10 - 1 - 1 - 1 - 1$$

$$\boxed{Performance(X) = 16}$$

Agent Y

Cleaning actions:

- Step 1 = **+10**
- Step 3 = **+10**

No movement occurs after both squares are known clean.

$$Performance(Y) = 10 + 10$$

$$\boxed{Performance(Y) = 20}$$

(e) Interpretation

Agent Y is the more rational agent for this environment.

At step 4, both agents receive the identical percept:

$$[B, Clean]$$

However, they behave differently because their internal structures are different.

Agent X is a **simple reflex agent**. It only considers the current percept. Since the current square is clean, it follows its rule:

If current square is clean, move to the other square.

Therefore, X moves Left.

Agent Y is a **model-based reflex agent**. It maintains an internal state containing squares that it knows to be clean. At step 4, it already knows:

$$\{A, B\} = \text{Clean}$$

Therefore, its rule says that if both squares are known clean, it performs **NoOp**.

Thus:

$$\boxed{X = 16,\ Y = 20}$$

So, **Y is rational** because it chooses the action that maximizes the expected performance based on its percepts and internal knowledge.

A **rational agent is not necessarily omniscient**. A rational agent chooses the best action based on the information available to it. An omniscient agent would know the actual future outcomes and all relevant information beforehand. Therefore, rationality means **doing the best possible action given the available information**, not knowing everything.

Question 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

Topic: Foundations and history of AI, problem-solving agents, problem formulation | 15 minutes

Part A. Complete the table by classifying each system or idea under one of the four approaches to AI (thinking humanly, acting humanly, thinking rationally, acting rationally) and naming the era or milestone of AI history to which it belongs.

(a) Complete the table. **[8 Marks]**

System / Idea	Approach to AI	Era / Milestone
Turing Test proposed (1950)		
General Problem Solver (Newell and Simon, 1957)		
Aristotle's syllogisms and formal logic		
An agent that does the right thing to maximise expected performance		

Part B. A problem-solving agent has two jugs: X of capacity 4 litres and Y of capacity 3 litres, both initially empty and without measuring marks. It may fill a jug from a tap, empty a jug on the ground, or pour one jug into the other until the source is empty or the target is full. The goal is to have exactly 2 litres in jug X.

(b) Formulate the problem: state representation, initial state, actions, transition model, goal test and path cost. **[6 Marks]**

(c) Generate all successor states of (0, 0) and of (4, 0) (leave out actions that leave the state unchanged). **[3 Marks]**

(d) How many states does the state space contain? Write a shortest solution as a sequence of states and give its path cost. **[3 Marks]**

(e) Interpret: describe the four phases (goal formulation, problem formulation, search, execution) of a problem-solving agent for this task, and explain why the agent can execute the found solution without looking at new percepts. **[5 Marks]**

QUESTION 2: ANSWER

(a) Four Approaches to AI

System / Idea	Approach to AI	Era / Milestone
Turing Test proposed (1950)	Acting humanly	1950 – Turing Test
General Problem Solver – Newell and Simon (1957)	Thinking humanly	1950s – Early symbolic AI / cognitive simulation
Aristotle's syllogisms and formal logic	Thinking rationally	Ancient foundations of formal logic
Agent that does the right thing to maximise expected performance	Acting rationally	Modern rational-agent approach

Four approaches

1. **Thinking humanly** – machines attempt to imitate human thought processes.
2. **Acting humanly** – machines attempt to behave like humans.
3. **Thinking rationally** – machines use logical reasoning to reach correct conclusions.
4. **Acting rationally** – machines choose actions that maximize expected performance.

(b) Problem Formulation

The two jugs have capacities:

$$X = 4L,\ Y = 3L$$

Initial state:

$$(0, 0)$$

Goal:

$$X = 2$$

1. State Representation

Represent a state as:

$$(x, y)$$

where:

- x= amount of water in 4-L jug X
- y= amount of water in 3-L jug Y

Example:

$$(3, 2)$$

means X contains 3 L and Y contains 2 L.

2. Initial State

$$\boxed{(0, 0)}$$

Both jugs are empty.

3. Actions

The agent can:

- Fill X
- Fill Y
- Empty X
- Empty Y
- Pour X → Y
- Pour Y → X

4. Transition Model

The transition model describes the resulting state after performing any valid action.

For example:

$$(4, 0) \xrightarrow{X \to Y} (1, 3)$$

5. Goal Test

The goal is achieved when jug X contains exactly 2 litres:

$$\boxed{x = 2}$$

6. Path Cost

Assume every action has cost 1.

$$\boxed{\text{Path Cost} = \text{Number of actions}}$$

(c) Successor States

From (0,0)

Possible useful actions:

Fill X

$$(0,0) \rightarrow (4,0)$$

Fill Y

$$(0,0) \rightarrow (0,3)$$

Therefore:

$$\boxed{Successors(0,0) = \{(4,0),(0,3)\}}$$

From (4,0)

Empty X

$$(4,0) \rightarrow (0,0)$$

Pour X → Y

Y has 3 L capacity, so 3 L moves from X to Y:

$$(4,0) \rightarrow (1,3)$$

Therefore:

$$\boxed{Successors(4,0) = \{(0,0),(1,3)\}}$$

(d) State Space and Shortest Solution

X can contain:

$$0,1,2,3,4$$

Y can contain:

$$0,1,2,3$$

Therefore:

$$5 \times 4 = 20$$

$$\boxed{\text{State Space} = 20}$$

Shortest solution

$$(0,0)$$

Fill X:

$$(4, 0)$$

Pour X → Y:

$$(1, 3)$$

Empty Y:

$$(1, 0)$$

Pour X → Y:

$$(0, 1)$$

Fill X:

$$(4, 1)$$

Pour X → Y:

$$\boxed{(2, 3)}$$

The goal $X = 2$is achieved.

Path

$$\boxed{(0, 0) \rightarrow (4, 0) \rightarrow (1, 3) \rightarrow (1, 0) \rightarrow (0, 1) \rightarrow (4, 1) \rightarrow (2, 3)}$$

Number of actions:

$$\boxed{6}$$

Therefore, path cost:

$$\boxed{6}$$

(e) Four Phases of a Problem-Solving Agent

1. Goal Formulation

The agent decides what it wants to achieve.

$$\boxed{\text{Goal: Obtain exactly 2 L in jug X}}$$

2. Problem Formulation

The agent defines:

- Initial state = (0,0)

- Actions = Fill, Empty and Pour
- Goal test = X contains 2 L
- Path cost = number of actions

3. Search

The agent searches the state space to find a sequence of actions that reaches the goal.

One solution is:

$$(0,0) \rightarrow (4,0) \rightarrow (1,3) \rightarrow (1,0)$$
$$\rightarrow (0,1) \rightarrow (4,1) \rightarrow (2,3)$$

4. Execution

The agent executes the actions in the obtained sequence.

The agent does not need new percepts because the problem is deterministic and the effects of each action are known in advance. Therefore, after finding the correct sequence, it can execute the plan directly.

Question 3: Searching for Solutions – Uninformed Search Strategies

Topic: Searching for solutions, BFS, DFS, uniform-cost search | 15 minutes

Consider the weighted undirected graph below. The start node is S and the goal node is G. Use graph search with an explored set; expand neighbours alphabetically, test the goal when a node leaves the frontier, and break ties alphabetically. BFS and DFS ignore edge costs while searching, but path costs are measured with them.

Edge	Cost	Edge	Cost
S – A	1	B – D	1
S – B	4	C – G	6
A – C	2	D – G	2
A – D	5		

(a) Perform BFS from S (queue initially [S]). Do not add a node that is already explored or already in the queue. **[6 Marks]**

Iteration	Node Dequeued	Queue after Expansion	Explored Set
1			
2			
3			
4			
5			
6			

(b) Perform DFS from S using a stack (initially [S]); push children so that the alphabetically first child is popped first. Write the stack with its top on the left. **[5 Marks]**

Iteration	Node Popped	Stack after Pushing Children
1		
2		
3		
4		

(c) Perform uniform-cost search from S. Write queue entries as Node:g(parent) and replace an entry whenever a cheaper path to a frontier node is found. **[7 Marks]**

Iteration	Node Expanded	g(n)	Priority Queue after Expansion
1			
2			

Iteration	Node Expanded	g(n)	Priority Queue after Expansion
3			
4			
5			
6			

(d) Complete the comparison table. **[4 Marks]**

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
BFS				
DFS				
UCS				

(e) Interpret: explain why UCS removed as many nodes as BFS yet found a cheaper path, why DFS needed the fewest nodes here but cannot be trusted in general, and why an explored set is needed in graph search (use iteration 3 of BFS as an example). **[3 Marks]**

QUESTION 3: ANSWER

(a) BFS

Initial:

$$Queue = [S]$$

Dry Run

Iteration	Node Dequeued	Queue after Expansion	Explored Set
1	S	[A, B]	{S}
2	A	[B, C, D]	{S, A}
3	B	[C, D]	{S, A, B}
4	C	[D, G]	{S, A, B, C}
5	D	[G]	{S, A, B, C, D}
6	G	[ ]	{S, A, B, C, D, G}

At iteration 6, G is removed from the frontier, so the goal test succeeds.

BFS solution path

$$\boxed{S \rightarrow A \rightarrow C \rightarrow G}$$

Path cost:

$$1 + 2 + 6 = 9$$

$$\boxed{Cost = 9}$$

(b) DFS

Initial stack:

$$[S]$$

The stack has its **top on the left**.

Children are pushed in reverse alphabetical order so that the alphabetically first child is popped first.

Iteration	Node Popped	Stack after Pushing Children
1	S	[A, B]
2	A	[C, D, B]

Iteration	Node Popped	Stack after Pushing Children
3	C	[G, D, B]
4	G	[D, B]

Goal G is reached at iteration 4.

DFS solution

$$\boxed{S \to A \to C \to G}$$

Path cost:

$$1 + 2 + 6 = 9$$

$$\boxed{Cost = 9}$$

(c) Uniform-Cost Search

Initial priority queue:

$$[S: 0]$$

Iteration 1

Expand S:

$$A: 1(S),\ B: 4(S)$$

PQ:

$$[A, : 1(S) B: 4(S)]$$

Iteration 2

Expand A, $g(A) = 1$:

$$C = 1 + 2 = 3$$

$$D = 1 + 5 = 6$$

PQ:

$$[C, : , 3(A) B: 4(S) D: 6(A)]$$

Iteration 3

Expand C, $g(C) = 3$:

$$G = 3 + 6 = 9$$

PQ:

$$[B, :, 4(S)D: 6(A)G: 9(C)]$$

Iteration 4

Expand B, $g(B) = 4$:

$$D = 4 + 1 = 5$$

Since 5 is cheaper than the existing cost 6, replace D's entry.

PQ:

$$[D, : 5(B)G: 9(C)]$$

Iteration 5

Expand D, $g(D) = 5$:

$$G = 5 + 2 = 7$$

Replace the previous G cost 9.

PQ:

$$[G: 7(D)]$$

Iteration 6

Expand G:

$$\boxed{g(G) = 7}$$

Goal reached.

UCS Table

Iteration	Node Expanded	g(n)	Priority Queue after Expansion
1	S	0	[A:1(S), B:4(S)]
2	A	1	[C:3(A), B:4(S), D:6(A)]
3	C	3	[B:4(S), D:6(A), G:9(C)]
4	B	4	[D:5(B), G:9(C)]
5	D	5	[G:7(D)]
6	G	7	[ ]

UCS solution path

$$\boxed{S \rightarrow B \rightarrow D \rightarrow G}$$

Path cost:

$$4 + 1 + 2 = 7$$

$$\boxed{Cost = 7}$$

(d) Comparison Table

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
BFS	S → A → C → G	9	6	No
DFS	S → A → C → G	9	4	No
UCS	S → B → D → G	7	6	Yes

(e) Interpretation

UCS removed the same number of nodes as BFS, but found a cheaper path because BFS considers the number of edges, whereas UCS considers the accumulated path cost $g(n)$. BFS selected:

$$S \rightarrow A \rightarrow C \rightarrow G$$

with cost:

9

UCS found:

$$S \rightarrow B \rightarrow D \rightarrow G$$

with cost:

7

DFS removed only 4 nodes in this particular graph because its alphabetical traversal reached G quickly. However, DFS cannot be trusted to find the optimal solution because it may follow a deep or expensive path before finding the goal.

An **explored set** prevents repeated expansion of already explored nodes. For example, during BFS, when B is expanded, D is already in the queue, so another copy of D is not added. This prevents duplicate states and unnecessary search.

Question 4: Search with Partial Information – Belief States

Topic: Problem formulation, search with partial information (sensorless and percept-based belief-state search) | 15 minutes

A vacuum agent lives in a two-square world (Left square, Right square), each square being Clean (C) or Dirty (D). Actions: Left, Right and Suck (Left in the left square and Right in the right square leave the agent where it is; Suck cleans the current square). Physical states are written (Agent, Left square, Right square):

State	Description	State	Description
S1	(L, D, D)	S5	(L, C, D)
S2	(R, D, D)	S6	(R, C, D)
S3	(L, D, C)	S7	(L, C, C)
S4	(R, D, C)	S8	(R, C, C)

(a) Initially the agent has no sensors and does not know its location or the dirt status. Formulate the problem over belief states: initial belief state, actions, goal test and path cost. **[3 Marks]**

(b) Apply the action sequence [Right, Suck, Left, Suck] and write the belief state after each action. **[6 Marks]**

Step	Action	Belief State after Action
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}
1	Right	
2	Suck	
3	Left	
4	Suck	

(c) Now the agent has a local sensor that reports [location, status of the current square]. Starting again from the initial belief state, the first percept is [Left, Dirty]; the agent then performs Suck and perceives [Left, Clean]; it then performs Right and perceives [Right, Dirty]. Fill in the belief state after each action (prediction) and after each percept (update). **[8 Marks]**

Step	Action	After Action (Predicted)	Percept	After Percept (Updated)
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}	[Left, Dirty]	
1	Suck		[Left, Clean]	
2	Right		[Right, Dirty]	

(d) How many different belief states are possible when there are 8 physical states? Why does this make the problem harder than searching over physical states? **[3 Marks]**

(e) Interpret: explain why the percepts in part (c) shrink the belief state, how this differs from the sensorless case in part (b), and why the sensorless plan can still be called rational. **[5 Marks]**

QUESTION 4: ANSWER

(a) Initial Belief-State Problem

Initial belief state

The agent does not know its location or dirt status.

$$\boxed{B_0 = \{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}}$$

Actions

$$\boxed{\{L, e, ftRightSuck\}}$$

Goal Test

All physical states in the belief state must represent a clean environment:

$$\boxed{B \subseteq \{S, 7S8\}}$$

Path Cost

Each action has cost 1.

$$\boxed{\text{Path Cost} = \text{Number of actions}}$$

(b) Sensorless Belief-State Dry Run

Initial belief:

$$B_0 = \{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}$$

Step 1 – Right

After Right, every state moves to R:

$$\boxed{B_1 = \{S, 2, S, 4S6S8\}}$$

Step 2 – Suck

The right square is cleaned in every state:

$$S2 \rightarrow S6$$

$$S4 \rightarrow S8$$

$$S6 \rightarrow S6$$

$$S8 \rightarrow S8$$

Therefore:

$$\boxed{B_2 = \{S, 6S8\}}$$

Step 3 – Left

Both possible states move to L:

$$S6 \rightarrow S5$$

$$S8 \rightarrow S7$$

Therefore:

$$\boxed{B_3 = \{S, 5S7\}}$$

Step 4 – Suck

The left square is cleaned:

$$S5 \rightarrow S7$$

$$S7 \rightarrow S7$$

Therefore:

$$\boxed{B_4 = \{S7\}}$$

Final table

Step	Action	Belief State After Action
0	Start	{S1, S2, S3, S4, S5, S6, S7, S8}
1	Right	{S2, S4, S6, S8}
2	Suck	{S6, S8}

Step	Action	Belief State After Action
3	Left	{S5, S7}
4	Suck	{S7}

Thus, after the sequence:

$$\boxed{[R, i, g, htSuckLeftSuck]}$$

the agent reaches the completely clean state **S7** regardless of the initial physical state.

(c) Sensor-Based Belief-State Updates

Initial belief:

$$B_0 = \{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}$$

First percept:

$$[L, eftDirty]$$

This eliminates every state where the agent is not in L or where L is clean.

Therefore:

$$\boxed{B_0^{updated} = \{S, 1S3\}}$$

Step 1 – Suck

Predicted belief:

$$S1 \rightarrow S5$$
$$S3 \rightarrow S7$$

Therefore:

$$\boxed{\text{Predicted} = \{S, 5S7\}}$$

Percept:

$$[L, eftClean]$$

Both states match this percept:

$$\boxed{\text{Updated} = \{S, 5S7\}}$$

Step 2 – Right

Prediction:

$$S5 \rightarrow S6$$
$$S7 \rightarrow S8$$

Therefore:

$$\boxed{\text{Predicted} = \{S, 6S8\}}$$

Percept:

$$[R, ightDirty]$$

Only S6 has the right square dirty.

Therefore:

$$\boxed{\text{Updated} = \{S6\}}$$

Complete table

Step	Action	After Action – Predicted	Percept	After Percept – Updated
0	Start	{S1,S2,S3,S4,S5,S6,S7,S8}	[Left, Dirty]	{S1,S3}
1	Suck	{S5,S7}	[Left, Clean]	{S5,S7}
2	Right	{S6,S8}	[Right, Dirty]	{S6}

(d) Number of Possible Belief States

There are 8 physical states.

A belief state is a subset of the physical-state set.

Therefore, the number of possible belief states is:

$$2^8$$

$$\boxed{256}$$

This makes the problem harder because the belief-state space grows **exponentially** with the number of physical states.

Instead of searching only 8 physical states, the agent may need to consider up to:

$$\boxed{256 \text{ belief states}}$$

(e) Interpretation

In part (c), the precepts provide additional information about the actual state of the environment. Therefore, each percept eliminates physical states that are inconsistent with the observation.

For example, the initial percept:

$$[L, eftDirty]$$

reduces the belief state from:

$$\{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}$$

to:

$$\{S, 1S3\}$$

After performing Suck, the predicted states are:

$$\{S, 5S7\}$$

The percept:

$$[L, eftClean]$$

confirms both states, so the belief remains:

$$\{S, 5S7\}$$

After Right, the prediction is:

$$\{S, 6S8\}$$

The percept:

$$[R, ightDirty]$$

eliminates S8 because its right square is clean. Therefore:

$$\boxed{\{S6\}}$$

Difference from sensorless search

In the **sensorless case**, the agent receives no percepts, so it must maintain all physical states consistent with its action history.

In the **sensor-based case**, each percept provides information and therefore shrinks the belief state.

The sensorless plan can still be called **rational** because rationality depends on choosing the best action based on the information available. Even without sensors, the sequence:

$$[R, i, g, htSuckLeftSuck]$$

guarantees that all possible initial states reach the clean state S7. Therefore, the plan achieves the goal despite uncertainty.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Accuracy of Solution Path	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Q. No. / Criteria	Understanding of Search Problem	State Expansion and Dry Run Execution	Application of Search Algorithm	Accuracy of Solution Path	Accuracy of Solution Path	Sub. Total (20)	Total %
1							
2							
3							
4							
Total							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____